

HOLY ANGEL UNIVERSITY

School of Computing
Holy Angel University
Angeles City

Final Project: DateMate AI

A Primary Course Requirement in Applications Development and Emerging Technologies - 6ADET



Submitted by
Marimla, Allain Stephanie S.

Section:
CS - 301

Submitted to:
Mr. Tjakoen A. Stolk
6-ADET Professor

September 20, 2026
Date of Submission

HOLY ANGEL UNIVERSITY

1. Proposal, version 2

Open your prelim proposal next to this. Work through it section by section., change what is now wrong, and fill in the two new sections at the bottom.

App name

DateMate AI (unchanged)

The problem, in one sentence

Re-read what you wrote. Does it still describe a real, specific situation? If you have drifted toward “an app for students”, fix it now.

College students and young working couples spend more time deciding where to eat or what to do than actually enjoying the date because they can have different preferences, budgets, and schedules. The apps they usually use, such as Google Maps, TikTok, Facebook, and Instagram, mostly show general suggestions instead of helping both people find an option that fits them.

Who is this for

Primary user: College students and young professionals who are in a relationship and regularly go on dates but have a hard time agreeing on where to eat or what activity to do.

Secondary user: Couples in long-distance relationships who want to plan dates before meeting and save places they both want to visit.

What they do today instead: Search Google Maps, TikTok, Facebook, or Instagram for date ideas, send links back and forth over chat, and keep comparing options until one person gives in.

HOLY ANGEL UNIVERSITY

Core features (MVP)

You now have built m4a4 and m4a5. You know roughly what one screen costs you in hours. Re-score your own feature list against that.

#	Feature	Still in the MVP?	Flutter pieces it needs	Honest estimate
1	Couple Profile Setup	Keep	TextField, DropdownButton, ElevatedButton, form validation, Firestore write on submit	5 hours.
2	AI Date Recommendation	Keep	TextField / Slider, DropdownButton, FutureBuilder, ListView.builder, CircularProgressIndicator	10 hours
3	Can't Decide Picker	Keep	ElevatedButton, Dart Random, showDialog or AnimatedSwitcher	2 hours
4	Date Bucket List (All / Pending / Completed, with rating + note on completion)	Keep	TabBar, TabBarView, ListView.builder, Dismissible or a status toggle, showModalBottomSheet, Card	8 hours

Total estimated MVP development time: 25 hours.

I decided to keep these four features because they are the main parts needed for the app to solve its

HOLY ANGEL UNIVERSITY

original problem. The Date History feature from my preliminary proposal is now included in the Completed section of the Bucket List instead of being a separate feature and screen.

Stretch goals

1. Weather-based date recommendations (e.g., suggest indoor activities instead of an outdoor picnic on a rainy day).
2. Anniversary and special-occasion reminders that trigger a personalized suggestion automatically.
3. Google Maps integration for turn-by-turn directions and travel time from a saved suggestion.

NEW: How my app saves data

The question from page 1 — if two different people install my app, should they see the same data?

Yes. DateMate AI is designed for couples, so both partners should be able to use the same couple profile from their own devices. If one partner changes their preferences or adds a restaurant to the Bucket List, the other partner should be able to see the updated information without having to enter it again.

Because the app needs to share data between two devices, saving everything only on one device would not be enough. I therefore need cloud storage for the main app data.

Roughly how many records in a realistic week of use:

For one couple, I expect around:

- 1 couple profile
- 5–10 generated date suggestions
- 5–15 Bucket List items

This would be around 20–35 records per couple in a typical week. The amount of data is relatively small, so storage capacity is not the main concern for this project.

HOLY ANGEL UNIVERSITY

My choice:

Firebase, using Cloud Firestore and Firebase Authentication.

Why this one and not the others:

I chose Firebase because the app needs the same information to be available on both partners' devices. shared preferences and local databases would keep the information on one device, so they would not work well for the shared couple profile.

I also considered Supabase. Supabase would be useful because it uses PostgreSQL and would make traditional database queries easier. However, I decided to use Firebase because Firestore has realtime listeners that fit the Bucket List feature well. Both partners can receive updates when something is added or changed without having to manually refresh the screen. The tradeoff is that I will have to work with Firestore's document structure instead of using a regular SQL database.

What I save, concretely:

Couple Profile

couples/{coupleId}

Fields:

- coupleId
- partnerAUId
- partnerBUId
- foodPreferences
- activityPreferences
- budgetRange
- preferredLocations

Date Suggestions

couples/{coupleId}/suggestions/{suggestionId}

HOLY ANGEL UNIVERSITY

Fields:

- suggestionId
- dateType
- place
- budget
- category
- dateGenerated

Bucket List

couples/{coupleId}/bucketList/{itemId}

Fields:

- itemId
- name
- category
- status
- dateAdded
- rating
- note
- dateCompleted

The rating, note, and completion date are only used once a Bucket List item is completed.

Have I tried it yet:

Yes, I did the one-hour Firebase test by adding `firebase_core` and `cloud_firestore`, writing a hardcoded document to a test collection, restarting the app, and reading the document back. This confirmed that the basic Firebase read and write process was working and that flutterfire configure was set up correctly.

I have not fully tested the two-device part yet. My next test is to use two test accounts connected to the same couple and check if a change made on one device appears on the other through Firestore's realtime listener.

HOLY ANGEL UNIVERSITY

NEW: One thing I want to add that the course did not teach

The feature I would like to add is **Google Maps integration for directions and travel time from a saved suggestion.**

- **Package:** google_maps_flutter, with geolocator for the user's current location.
- **Where it runs:** It can run on Android/iOS. Flutter web support for the embedded map needs additional setup.
- **Web fallback:** On web, instead of showing an embedded map, the "Get Directions" button can open the location in Google Maps in another browser tab. This allows the rest of the app to continue working normally in a browser.
- **How I will show the real feature:** The embedded map can be demonstrated through a phone screen recording in the final presentation.
- **Core or stretch:** This is a **stretch goal** because DateMate AI can still solve its main problem without the map. The main purpose of the app is to help couples choose and remember date ideas, while directions are an additional convenience.

How my project runs when someone else opens it

Keeping the device_preview wrapper:

Yes. I will keep the device_preview wrapper because it makes it easier to check the different screens and allows the project to be viewed without always needing a physical phone.

Runs in a browser with flutter run -d web-server, every screen reachable:

Not yet fully. The five MVP screens are planned to be reachable in the browser, but I still need to complete the full start-to-finish web test, especially the Firebase Authentication, shared-data flow, and security rules. The AI recommendation API uses HTTP, so that part can work on the web. The Maps stretch feature will use the web fallback described above.

HOLY ANGEL UNIVERSITY

Hardware-dependent features degrade instead of crashing:

Not applicable to the MVP. The MVP does not require the camera or other device sensors. The Maps feature is the main feature that needs platform-specific handling, and it will use the browser link-out fallback instead of causing the app to crash.

Public repository — keys, passwords, or real personal data:

My project will be kept in a **public GitHub repository under my own account** rather than the course organization. The AI API key is something I need to protect because it may be connected to a billable service. I will keep it in a local .env file that is ignored by Git and use a GitHub repository secret for deployment if needed.

The Firebase configuration file can identify the Firebase project, but access to the actual data will be controlled through Firebase Authentication and Firestore security rules. I will make sure the Firestore rules are changed from test mode before using real accounts.

I will not put real users' personal information, passwords, or private couple data in the repository. Only test or dummy data will be used in the project files.

Repository: github.com/allainstephanie26-alt/datemate-ai

Data the app remembers

Thing	Fields	Where it is saved
Couple Profile	coupleId, partnerAUId, partnerBUId, food preferences, activity preferences, budget range, preferred locations	Firestore: couples/{coupleId}
Date Suggestion	suggestionId, dateType, place/activity, budget, category, date generated	Firestore: couples/{coupleId}/suggestions/{suggestionId}
Bucket List Item	itemId, name, category, status	Firestore:

HOLY ANGEL UNIVERSITY

	(pending/completed), date added, rating, note, date completed (once completed)	couples/{coupleId}/bucketList/{itemId}
--	--	--

Screens

The revised app will have 5 screens:

1. Login / Sign Up
2. Couple Preferences — includes the partner-linking code and shared preferences
3. Home Dashboard
4. AI Date Recommendation — includes the "Can't Decide? Pick for us" action
5. Bucket List — contains All, Pending, and Completed tabs; the Completed tab also serves as the app's date history

Risks, revised

The risk I named last time:

My original risk was building the recommendation system so it could properly compare two people's preferences and recommend something that both would enjoy. After working on the previous modules, I feel that the UI part of this feature is less risky because I now have more experience using FutureBuilder and ListView.builder to display API results.

However, the actual recommendation logic is still a risk. The AI needs to consider both people's preferences instead of favoring only one person's choices. This will need testing with different combinations of preferences.

First step: I will create two hard-coded preference sets that intentionally conflict with each other, such as one person preferring vegetarian and low-budget options while the other prefers steak and has a higher budget. I will test the AI prompt and check whether the results actually consider both users.

Target date: August 27, 2026.

HOLY ANGEL UNIVERSITY

A new risk I did not see before:

The new risk is the Firebase shared-data and partner-linking setup. Since two people need to access the same couple profile, I need to make sure the two accounts are correctly connected to the same couple and that changes to the Bucket List appear on both devices.

There is also a security risk because Firebase projects can initially use test-mode rules. I need to replace those rules with proper authentication-based rules before using the app with real accounts.

First step: I will test the partner-linking process with two test accounts and make sure both accounts can access the same couple document and see changes to the Bucket List.

Target date: August 28, 2026.

For the Firebase security rules, I will also write and test the authentication-based rules before the test-mode period expires, and I will check them again after Firebase Authentication is connected.

What changed, and why

Section	Prelim said	Now says	Why it changed
Screens	6 screens, with Date History as a separate screen	5 screens, with Date History included in the Completed tab of the Bucket List	After reviewing the amount of work involved in the screens, I realized that Date History is mainly another view of completed Bucket List items. Keeping it as a separate screen would add work without adding much to the main purpose of the app.
Core Features	5 features, including Date History	4 features, with date history included in Bucket List	Date History is now part of the Bucket List's Completed state, so it does not need to be treated as a separate

HOLY ANGEL UNIVERSITY

			core feature.
Persistence	No storage solution was finalized	Firebase Cloud Firestore + Firebase Authentication	The revised plan requires both partners to access the same data from different devices, so local-only storage would not be enough.
Data	Date History had its own separate data entry	Completed Bucket List items now contain rating, note, and completion date	This keeps the stored data consistent with the decision to combine Date History with the Bucket List.
Risks	Main concern was the recommendation system	Recommendation logic remains a risk, while Firebase synchronization and security were added as new risks	Building the earlier modules made the UI side of the recommendation feature more familiar, but the shared-data requirement introduced new technical risks that I need to test.
Extra feature	Google Maps was listed as a stretch goal	Google Maps remains a stretch goal with a web fallback and phone demonstration plan	I still want to add directions, but it is not necessary for the app to solve its main problem, so it remains outside the MVP.

Overall, the main purpose of DateMate AI has stayed the same. The main changes were made to reduce unnecessary screens, make the MVP more realistic, decide how shared data will be stored, and account for the technical issues I discovered while working on the project.

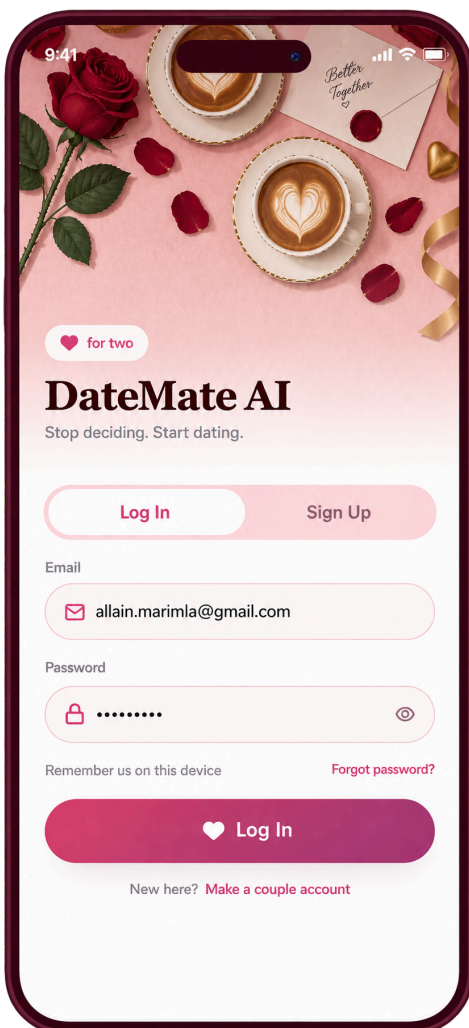
HOLY ANGEL UNIVERSITY

2. High-level mockup

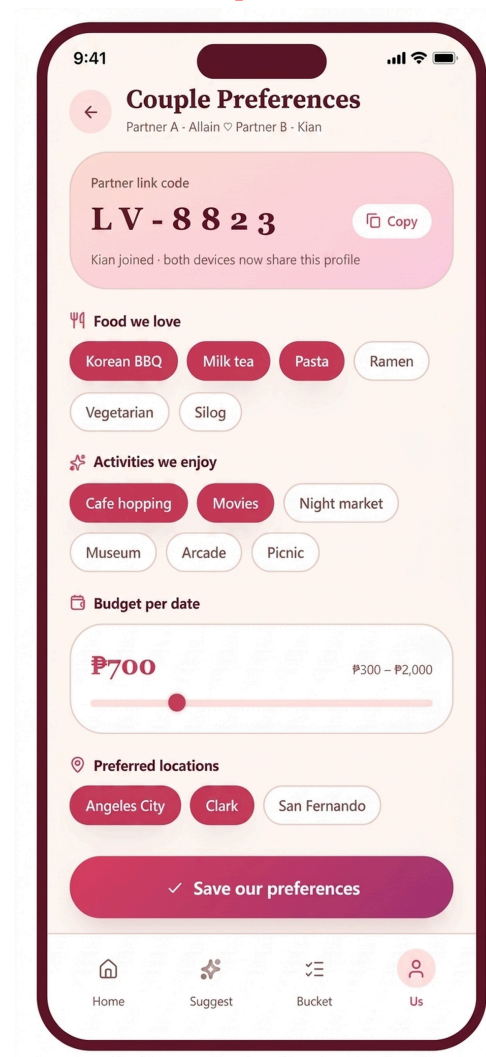
Your wireframes are done. You drew them for the prelim and they answered what goes where: boxes, labels, and the path from one screen to the next. Leave them alone. This is the next layer, not a redo. A mockup answers a different question: what will this actually look like? Same screens, same layout, but now with your real colors, your real type, real words instead of "label here", and icons where icons go. It is the picture you would show someone to explain your app without opening it. You are handing in images. A PDF of your screens, or PNGs, exported from whatever tool you like. Not a description of a mockup. The picture is the deliverable.

Step A and Step B: DateMate AI Five Screens

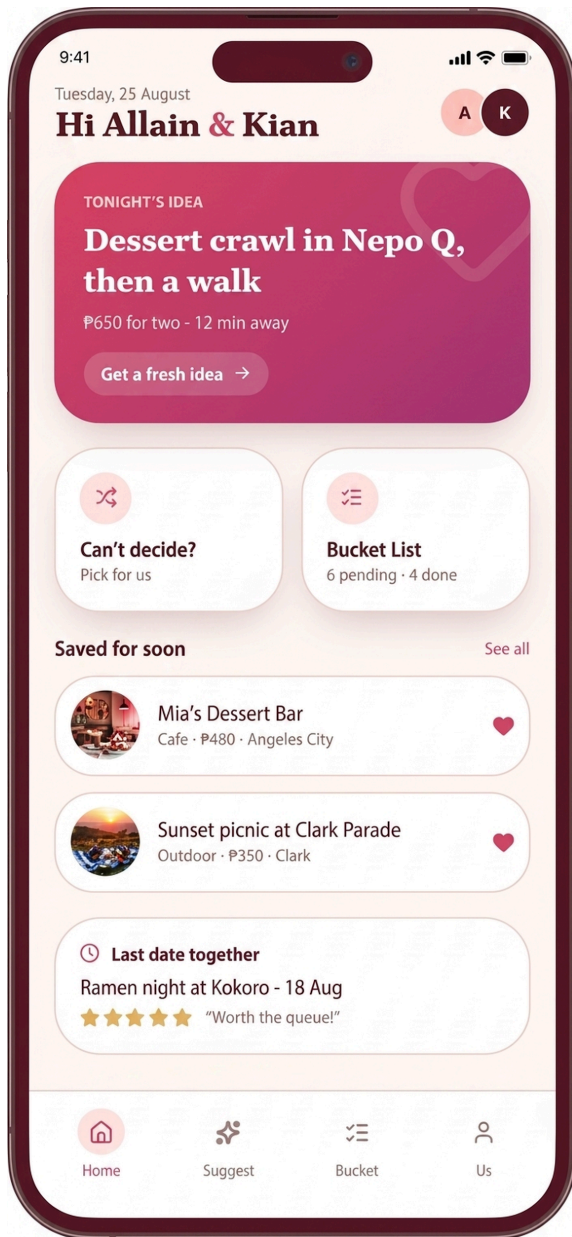
1. Login/Sign up



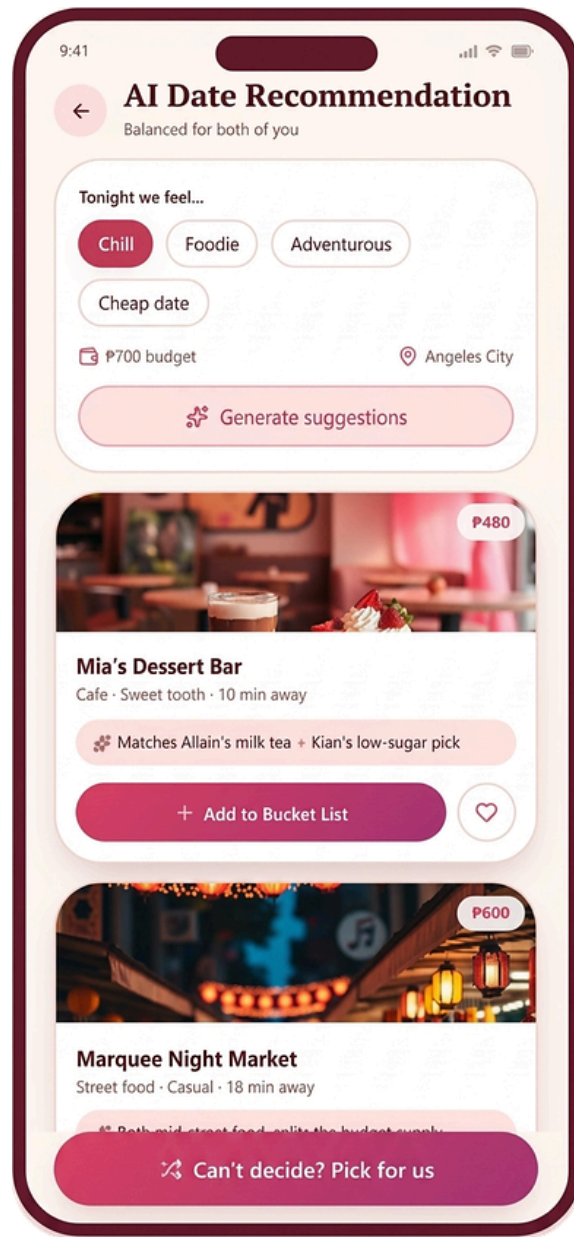
2. Couple Preferences



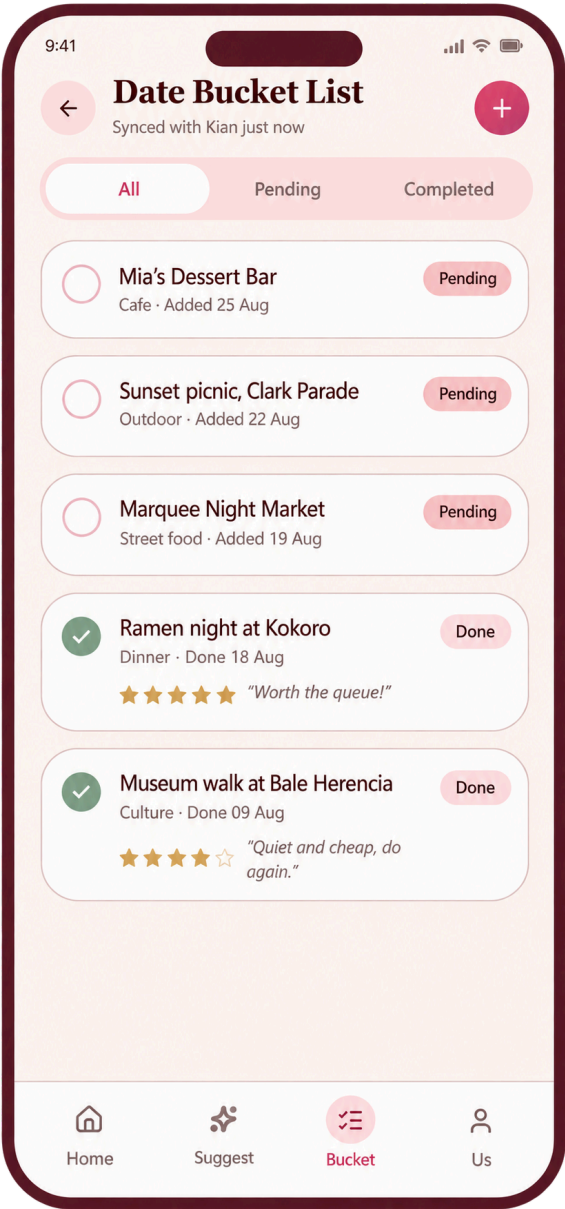
3. Home Dashboard



4. AI Date Recommendation



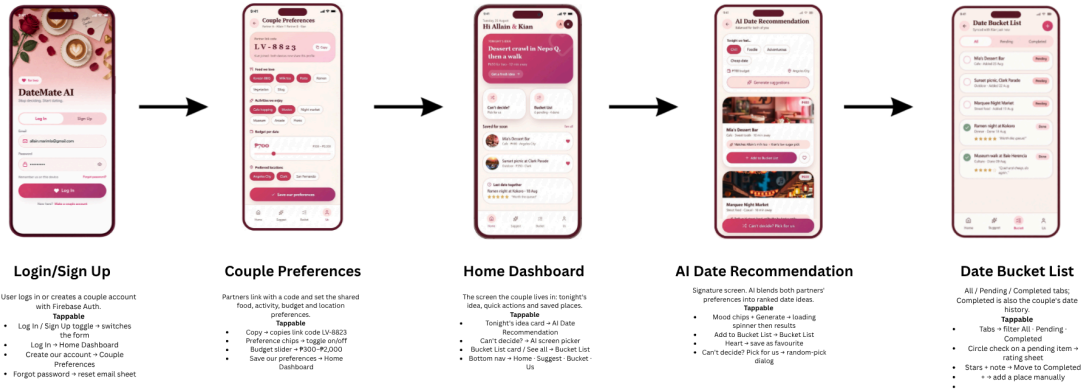
5. Date Bucket List



HOLY ANGEL UNIVERSITY

Step C and Step D: Label and In order

DateMate AI - High Level Mockup



Canva Link: <https://canva.link/4iqk312cbp0sfvp>

What Changed and Why

Your wireframes stay as they were. This section is about what you learned while painting them in, because that is where plans usually break.

Screen or element	The wireframe assumed	The mockup shows	What I will change
Couple Preferences	All four preference groups (food, activities, budget, locations) plus the Save button would fit on one screen without scrolling	With real chip labels and the slider drawn at full size, the Save button sits below the fold on a standard phone height	Keep Save our preferences pinned to the bottom of the screen instead of flowing with the content, so it's always reachable without hunting for it
Home Dashboard shortcuts	"Can't decide?" and "Bucket List" were two unrelated quick links in the wireframe	Painting them in, both turned out to be the exact same shape — icon circle, bold title, gray subtitle	Already folded this into the design system as a single ShortcutTile component instead of two one-off widgets, which is also why it shows up in the design system's components

HOLY ANGEL UNIVERSITY

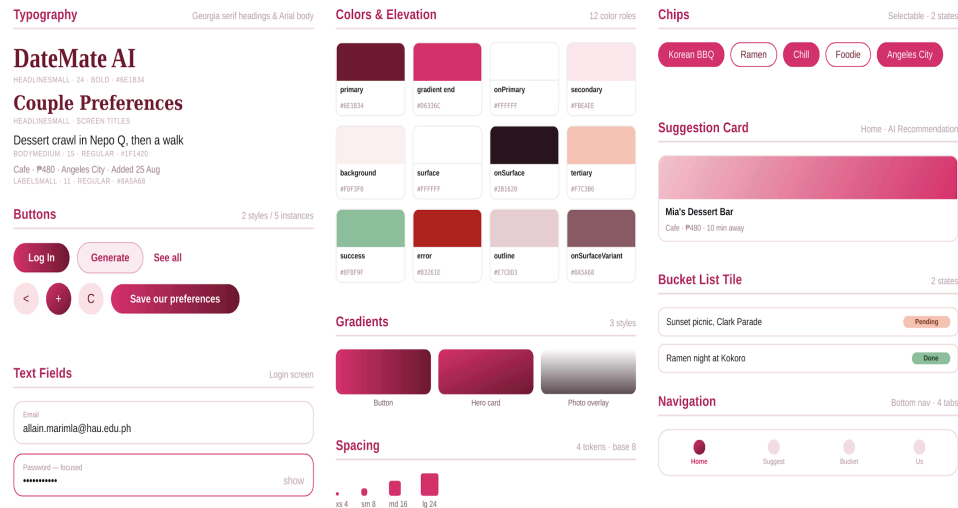
			list
AI Date Recommendation cards	Assumed three suggestion cards would fit above the "Can't decide?" button	With real photos, prices, and the "why it matches" tag at full size, only about 1.5 cards fit before the screen needs to scroll	Confirms the suggestion list needs ListView.builder with the "Can't decide? Pick for us" button pinned outside the scroll area, not placed after the last card



HOLY ANGEL UNIVERSITY

3. Design System, version 2

Last time you picked colors on paper. You have since written a real ThemeData in m4a3 and shipped two graded apps. This pass turns your design system into code you can paste.



Canva Link: <https://canva.link/4iqk312cbp0sfvp>

Step A: The palette, as a ColorScheme (10 min)

Write your palette as the Flutter object it becomes. Fill in real hex values.

```
final scheme = ColorScheme.fromSeed(
  seedColor: const Color(0xFF6750A4), // your primary
  brightness: Brightness.light,
);
```

ColorScheme.fromSeed generates a full, contrast-checked Material 3 palette from one color. Use it unless you have a reason not to, and if you do have a reason, write the full scheme out:

Role	Hex	Used for
primary	#6E1B34	Main buttons, active states, serif headings

HOLY ANGEL UNIVERSITY

onPrimary	#FFFFFF	Text and icons on primary/gradient buttons
secondary	#FBEAEE	Highlights, unselected pill background, tab track
surface	#FFFFFF	Cards, sheets, list tiles, text inputs
onSurface	#2B1620	Body text
error	#B3261E	Validation, destructive actions

Two checks the prelim did not ask for:

1. **Contrast.** *Put your body text color on your background color and look at it on a phone, outdoors if you can. If you generated with fromSeed, this is handled; if you hand-picked, check with a contrast tool (<https://webaim.org/resources/contrastchecker/>, aim for 4.5:1 on body text).*
 Body text (onSurface #2B1620) on background (#FDF3F0) $\approx 13.8:1$. Button text (onPrimary #FFFFFF) on primary (#6E1B34) $\approx 9.9:1$. Both clear the 4.5:1 minimum for body text.
2. *Dark mode: yes or no, decided now. It is one extra ColorScheme with brightness: Brightness.dark plus darkTheme: on your MaterialApp, and it is much cheaper to decide now than to retrofit after you have hardcoded Colors.white in fifteen places. If the answer is no, write "light only" and then never hardcode a color anywhere except this file.*
 Light only. Then all of the five screens use warm cream/pink surfaces; no dark variant is planned. And colors are only ever referenced through Theme.of(context).colorScheme, never hardcode a color.

Step B: The type scale, as a TextTheme (10 min)

Your style	Flutter slot	Size	Weight	Used for
Heading	headlineSmall	24	Bold	Screen titles ("Couple Preferences," "AI Date Recommendation") and the app name

HOLY ANGEL UNIVERSITY

Body	bodyMedium	15	Regular	Card titles, list-item names, chip labels, form values
Caption	labelSmall	11	Regular	Timestamps, prices, distances, hint text

Used by name, never by literal size:

Text('DateMate AI', style: [Theme.of\(context\).textTheme.headlineSmall](#))

That line is the whole point of a design system. Every TextStyle(fontSize: 24) you write inline is a decision you will have to find and change later. Custom fonts: google_fonts puts any Google font into your theme in one line, no asset files. See page 11. One font family is plenty.

Step C: Spacing, as constants

```
class AppSpacing {
  static const double xs = 4;
  static const double sm = 8;
  static const double md = 16;
  static const double lg = 24;
}
```

Base unit: 8

Screen edge padding: 16 (md)

Gap between list items: 16 (md) — Bucket List rows, "Saved for soon" cards

Gap between sections: 24 (lg) — between the "Tonight's idea" hero card and the shortcut tiles on Home

HOLY ANGEL UNIVERSITY

Step D: Component, as files

Component	File	Constructor Parameters	Appears on
PrimaryGradientButton	lib/widgets/primary_gradient_button.dart	String label, VoidCallback onPressed, IconData? icon	Login ("Log In"), Couple Preferences ("Save our preferences"), AI Recommendation ("Add to Bucket List," "Can't decide? Pick for us")
SelectableChip	lib/widgets/selectable_chip.dart	String label, bool selected, VoidCallback onTap	Couple Preferences (food/activity/location tags), AI Recommendation (mood tags)
DateSuggestionCard	lib/widgets/date_suggestion_card.dart	String title, String subtitle, String price, String imageUrl, VoidCallback onAddToBucket, VoidCallback onFavorite	Home ("Tonight's idea" hero + "Saved for soon" list), AI Recommendation (generated suggestion cards)
BucketListTile	lib/widgets/bucket_list_tile.dart	String name, String category, String status, DateTime date, double? rating, String? note, VoidCallback onToggle	Bucket List (All/Pending/Completed rows), Home ("Saved for soon" and "Last date together" rows)
AppBottomNavBar	lib/widgets/app_bottom_nav_bar.dart	int currentIndex, ValueChanged<int> onTap	Home, Couple Preferences, AI Recommendation, Bucket List (every screen except Login)

Every component takes data and callbacks only — none call `setState`. The parent screen owns the list of items and passes one item plus one callback into each tile; the tile renders it and reports the tap back up. All five are marked `const-constructible` wherever the caller's data allows it.

HOLY ANGEL UNIVERSITY

What changed, and Why

Element	Prelim said	Now says	Why it changed
Palette	Rose/maroon named as a mood, no hex values or roles	ColorScheme roles pulled from the shipped mockup: #6E1B34 primary, #D6336C gradient end, #FDF3F0 background, #FBEAEE secondary	Building m4a4/m4a5 forced me to assign a real hex to every widget instead of describing a mood
Type scale	"Elegant serif for headers, clean sans for body," no sizes given	headlineSmall 24/bold serif, bodyMedium 15/regular sans, labelSmall 11/regular sans, mapped to named slots	Wrote real TextStyles while building m4a5 and measured what the mockup actually uses at each size
Components	Not broken into files; only screen sketches existed	5 files with explicit constructor parameters	The Bucket List and Home screens repeat the same card and tile shapes, so componentizing them is required, not optional

HOLY ANGEL UNIVERSITY**4. Midterm Reflection Journal****A. Module 4: Making a Screen**

Module 4 was when I started getting used to making an actual screen instead of only writing Dart code. I see a widget as one part of the screen. It can be a text, button, image, or something that holds other widgets. The build method is where I put these widgets together. The widget tree started making more sense when I saw that widgets can be placed inside other widgets.

The layout part took me the longest to understand. I had problems with the position and spacing of things, especially when using Row, Column, Padding, and Expanded together. At first, I would just change the values until the screen looked correct. After doing the activities, I started understanding that each widget has a different purpose. Row is for putting things beside each other, Column is for putting things vertically, and Expanded helps a widget use the available space. I understood this more by actually seeing what happened when I changed the code.

M4A3 also helped me understand theming. Instead of setting the same colors and styles again on every screen, I can set the main design in one place. I think this will help with DateMate AI because I want the different screens to have a consistent design.

For M4A4, I was happy that I was able to make a screen that looked like an actual application. If I did it again, I would plan the layout more before starting because I spent time fixing the design while I was already coding.

B. Module 5: Making It Move

Module 5 helped me understand how an app responds to what the user does. I learned that changing a variable does not automatically change what is shown on the screen. setState is needed so Flutter knows that something changed and needs to be displayed again.

I also learned more about TextField and controllers. At first, I thought getting the text entered by the user would be simpler. I had to use the controller to get the value and then use it in the program.

ListView.builder also became clearer after using it in an activity. Instead of putting every item manually in a Column, the list can create the items based on the data. This is something I can use in the DateMate AI bucket list.

HOLY ANGEL UNIVERSITY

For navigation, Navigator.push takes me to another screen and pop takes me back. In M5A4, I understood the basic navigation but had some difficulty with passing information to the next screen. Doing the activity helped me understand how the screens can pass the needed information.

For M5A5 HAUDEx, I spent around 1-2hours. A lot of that time was spent testing and fixing problems. Some tests returned -1, so I had to go back through my code instead of assuming that everything was wrong. This made me realize that I should include debugging time when planning my final project.

C. What Clicked, and What Is Still Shaky

Navigation was one thing that finally clicked for me in M5A4. I already knew what push and pop did, but I understood them better when I actually had to move information between screens.

I still need more practice with a shared state between screens. I understand setState when the change is happening on one screen, but I am not yet completely comfortable when the same information needs to be used by several screens.

One bug I was proud of solving was from my HAUDEx activities. I started checking the code and output step by step instead of changing random parts of the program. An error I now recognize is the RenderFlex overflowed. I usually know to check whether the contents of a Row or Column are taking up more space than what is available.

D. How I Work

When I get stuck now, I usually read the error first and check the code around it. Before, I would sometimes keep changing things until they worked. I also use AI when I need help understanding an error or a concept. It can help me see what I should check, but I have also received code that did not work with my project. There were times when I could not explain some of the code it provided, so I learned that I still have to understand and test what I use.

I also left some activities closer to the deadline, which made debugging more stressful. For the final project, I want to start earlier and test each part as I build it.

For the final project, I want to keep the habit of testing while I build instead of waiting until the end. One habit I want to change is starting earlier, especially for activities that involve several screens or debugging.

E. Looking Forward

DateMate AI will use many of the things I learned in Module 5. The Couple Preferences screen needs user input, the AI Date Recommendation screen needs to respond to preferences, and the Date Bucket List needs to display and update saved items. Navigation will also be important because the five main screens need to work together.

The biggest gap right now is saving data. My previous activities mostly worked while the program was running, but DateMate AI needs user information, preferences, and saved dates to remain available.

By the end of the project, I want to be able to say that I can connect the UI, input, navigation, state, and saved data into one working application. Two weeks from now, I want the five main DateMate AI screens connected and the basic input, recommendation, and bucket-list flow working.